Localization and Navigation Methods for Materialenbuddy

Objective and scope

This document sets out the methods chosen so that Materialenbuddy can localize itself and navigate with and without a map.

The robot works in two separate modes: with a known map (localization with AMCL) and without a map or extending one (SLAM). Switching between modes is manual. Each section describes which method is used, why, and what is expected of it, so it can be implemented later without revisiting decisions.

Hardware and environment

The methods are designed for a Create 3 with a 2D LiDAR and a Raspberry Pi 5.

- **Base:** iRobot Create 3, firmware I.0.0 CycloneDDS (Iron), a circular differential-drive robot about 34 cm in diameter.
- **Communication:** `ROS_DOMAIN_ID` 21, namespace `/r2`, `rmw_cyclonedds_cpp`.
- **Navigation sensor:** 2D LiDAR
- **Compute:** onboard Raspberry Pi 5 with native ROS 2 Jazzy.
- **Development:** laptop with ROS 2 in WSL and RViz2.
- **Out of scope:** 3D LiDAR, direct motor access, external IMU, and floor sensors as an odometry source.

Odometry method

The starting point is the fused odometry the Create 3 already publishes, and the EKF filter remains an optional improvement.

- **Base source:** `/r2/odom` and the robot's own `odom -> base_link` TF, which already combine encoders, IMU, and optical flow.
- **EKF (`robot_localization`):** optional; it would fuse `/r2/odom` and `/r2/imu`. It is only adopted if it improves measurements compared with the native odometry.
- **TF rule:** there must be a single publisher of `odom -> base_link`. If the EKF takes over, the Create 3 is configured with `publish_odom_tfs: false`.
- **Expected result:** a continuous `odom -> base_link` without sudden jumps.

Mapping and localization method

The map is built with `slam_toolbox` and used with AMCL; the same mapping process allows it to be extended later.

- **Mapping:** `slam_toolbox` in asynchronous mode with the 2D LiDAR, so the load is manageable on the Raspberry Pi 5. Mapping always starts with the robot on the dock, which becomes the reference pose.
- **Two save formats:** the occupancy map (`.yaml/.pgm`) for AMCL and the serialized pose-graph to continue mapping. A `.pgm` alone cannot be used to extend the map.
- **Extension (lifelong):** the pose-graph is loaded and `slam_toolbox` keeps mapping from the dock pose.
- **Localization with a map:** `nav2_map_server` + `nav2_amcl` on the saved map, with the robot on the dock as the initial pose.
- **TF rule:** AMCL and `slam_toolbox` both publish `map -> odom`; since the modes are separate, only one is launched.

Manual mode switching method

The mode is chosen by hand before launching the system and changed by stopping one and starting the other; there is no automatic supervisor, and the two never run at the same time.

Mode	Active system	When it is used	Startup requirement
With map	`nav2_map_server` + `nav2_amcl`	Area already mapped	Saved `.yaml/.pgm` map and robot on the dock (known initial pose)
SLAM	Asynchronous `slam_toolbox` (lifelong to extend)	Area without a map or extending an existing map	To extend: serialized pose-graph and robot on the dock

1. **Stop:** halt the robot and fully shut down the active mode.
2. **Save (when leaving SLAM):** the `.yaml/.pgm` map for AMCL and the pose-graph to extend it later. Before switching modes, the robot returns to the dock.
3. **Start:** launch the other mode by selecting it in the launch file.
4. **Verify:** in map mode, check in RViz2 that the robot is correctly placed on the map and that `map -> odom` is stable before navigating.

Navigation method with Nav2

Nav2 works the same way in both modes; only the source of the map and of `map -> odom` changes.

- **Global costmap:** static map in AMCL mode, `slam_toolbox` map in SLAM mode.
- **Local costmap:** rolling window fed by the LiDAR for dynamic obstacles.
- **Global planner:** A* (`NavfnPlanner` with A* or `SmacPlanner2D`).
- **Local controller:** Regulated Pure Pursuit (RPP).
- **Robot parameters:** `robot_radius` close to 0.17 m and low initial speeds (linear ≤ 0.3 m/s).

- **Namespace:** the whole stack under `r2`, with `tf` and `tf_static` remapped to connect with `/r2/tf`, `/r2/odom`, and `/r2/cmd_vel`.
- **Safety:** the Create 3's cliff and bump reflexes are kept enabled.

Sources

- Create 3: ROS 2 API
- slam_toolbox (Jazzy)
- create3_republisher (Jazzy)